

SISTEMAS
DISTRIBUIDOS

Práctica 4.

Multiprocesamiento y multiservicios.

MICHELLE ANGUIANO RODEA

ESCOM

PROFESOR CHADWICK CARRETO
ARELLANO

GRUPO: 7CM6



Instituto
politécnico
nacional

ESCOM

1. Antecedentes (Introducción)

En el campo de los sistemas distribuidos, un servidor debe ser capaz no solo de aceptar múltiples conexiones concurrentes, sino también de ofrecer diferentes servicios y, en algunos casos, distribuir la carga entre varios servidores.

Este enfoque es fundamental en aplicaciones modernas como nubes de cómputo, balanceadores de carga, sistemas de mensajería, servidores web y aplicaciones de streaming.

Los conceptos principales aplicados en esta práctica son:

- Multiprocesamiento: uso de hilos y pools de ejecución para atender a múltiples clientes de forma simultánea.
- Multiservicios: un mismo servidor puede responder a distintos comandos o funcionalidades, como ECHO, TIME, MAYUS o SUM.
- Balanceo de carga: un Load Balancer distribuye las conexiones entrantes hacia distintos servidores backend, usando políticas como round-robin.
- Sincronización: uso de semáforos y secciones críticas para controlar el acceso a recursos limitados (simulación de “M:30 procesos en servicio”).

Este esquema se asemeja al funcionamiento real de sistemas como NGINX, HAProxy, servidores de bases de datos replicadas y microservicios en contenedores.

2. Problemática

La práctica planteó los siguientes retos:

1. Implementar un servidor balanceador que acepte conexiones y las reparta entre múltiples backends.
2. Desarrollar servidores backend multihilo, cada uno con diferentes servicios disponibles.
3. Garantizar la concurrencia controlada usando mecanismos de sincronización.
4. Permitir que los clientes interactúen con el sistema solicitando distintos servicios, sin importar a qué backend estén conectados.
5. Verificar que la arquitectura soporte múltiples clientes concurrentes sin errores de comunicación ni bloqueos.

3. Solución

La solución diseñada consistió en tres componentes principales:

- Load Balancer (LoadBalancer.java)
 - Escucha en el puerto principal (8080).
 - Usa una política round-robin para repartir conexiones entre los backends (9001, 9002, etc.).
 - Implementa “pipes” de entrada y salida para reenviar la comunicación transparente entre cliente y servidor.
- Servidores Backend (BackendServer.java)
 - Cada backend escucha en un puerto distinto (9001, 9002).
 - Atiende múltiples clientes mediante un pool de hilos (ExecutorService).
 - Expone varios servicios:
 - ECHO <mensaje> → devuelve el texto tal cual.
 - TIME → devuelve la hora actual.
 - MAYUS <mensaje> → devuelve el texto en mayúsculas.
 - SUM a b → devuelve la suma de dos enteros.
 - Usa un Semaphore(M) para simular el límite de recursos disponibles (ejemplo: M=3).
- Clientes (ClientMulti.java)
 - Se conectan al puerto del Load Balancer.
 - Envían comandos a través de una consola interactiva.
 - Reciben la respuesta del backend que les haya tocado según la política de reparto.

4. Implementación

La implementación se realizó en Java, con las siguientes clases:

1. LoadBalancer.java

- Crea un ServerSocket en el puerto 8080.

- Redirige las conexiones a los backends con round-robin.

```
import java.io.*;
import java.net.*;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class LoadBalancer {
    // Backends registrados (puedes agregar más)
    private static final List<InetSocketAddress> BACKENDS = List.of(
        new InetSocketAddress(hostname:"127.0.0.1", port:9001),
        new InetSocketAddress(hostname:"127.0.0.1", port:9002)
    );
    private static final AtomicInteger rr = new AtomicInteger(initialValue:0);

    Run | Debug
    public static void main(String[] args) throws Exception {
        int port = 8080;
        try (ServerSocket ss = new ServerSocket(port)) {
            System.out.println("[LB] Escuchando en " + port + " con " + BACKENDS.size() + " backends...");
            while (true) {
                Socket client = ss.accept();
                new Thread(() -> handle(client)).start();
            }
        }

        private static void handle(Socket client) {
            InetSocketAddress backend = pickBackend();
            try (client;
                Socket srv = new Socket(backend.getHostName(), backend.getPort());
            ) {
                // ...
            }
        }
    }
}
```

```
        InputStream cin = client.getInputStream();
        OutputStream cout = client.getOutputStream();
        InputStream sin = srv.getInputStream();
        OutputStream sout = srv.getOutputStream() {

            // Pipe bidireccional simple
            Thread up = pipe(cin, sout);
            Thread down = pipe(sin, cout);
            up.join(); // cuando el cliente cierra, terminamos
            srv.shutdownOutput();
            down.join();
        } catch (Exception e) {
            System.err.println("[LB] Error: " + e.getMessage());
        }
    }

    private static InetSocketAddress pickBackend() {
        int i = Math.floorMod(rr.getAndIncrement(), BACKENDS.size());
        return BACKENDS.get(i);
    }

    private static Thread pipe(InputStream in, OutputStream out) {
        Thread t = new Thread(() -> {
            try (in; out) {
                byte[] buf = new byte[8192];
                int n;
                while ((n = in.read(buf)) != -1) out.write(buf, 0, n);
            }
        });
        return t;
    }
}
```

```

        } catch (IOException ignore) {}
    });
    t.setDaemon(on:true);
    t.start();
    return t;
}
}

```

2. BackendServer.java

- Usa ExecutorService para manejar múltiples hilos.
- Limita la concurrencia mediante un Semaphore.
- Procesa peticiones con diferentes comandos.

```

import java.io.*;
import java.net.*;
import java.time.Instant;
import java.util.Locale;
import java.util.concurrent.*;

public class BackendServer {
    private final int port;
    private final ExecutorService pool;
    private final Semaphore slots; // sincronización: capacidad M

    public BackendServer(int port, int maxThreads, int maxInService) {
        this.port = port;
        this.pool = Executors.newFixedThreadPool(maxThreads);
        this.slots = new Semaphore(maxInService, fair:true);
    }

    public void start() throws IOException {
        try (ServerSocket ss = new ServerSocket(port)) {
            System.out.println("[BE " + port + "] listo. Threads=" +
                ((ThreadPoolExecutor)pool).getMaximumPoolSize() + " M=" + slots.availablePermits());
            while (true) {
                Socket s = ss.accept();
                pool.submit(() -> handle(s));
            }
        }
    }

    private void handle(Socket s) {

```

```

private void handle(Socket s) {
    String who = s.getRemoteSocketAddress().toString();
    System.out.println("[BE " + port + "] +" + who);
    try (s;
        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
        PrintWriter out = new PrintWriter(new OutputStreamWriter(s.getOutputStream()), autoFlush:true)) {

        out.println("Backend " + port + " OK. Comandos: ECHO, TIME, MAYUS, SUM. 'exit' para salir.");

        String line;
        while ((line = in.readLine()) != null) {
            if (line.equalsIgnoreCase(anotherString:"exit")) break;

            // Sección crítica Limitada (simula M:30 recursos)
            slots.acquire();
            try {
                out.println(dispatch(line));
            } finally {
                slots.release();
            }
        }
    } catch (Exception e) {
        System.err.println("[BE " + port + "] Err: " + e.getMessage());
    } finally {
        System.out.println("[BE " + port + "] -" + who);
    }
}

```

```

// Multiservicio
private String dispatch(String line) {
    String[] p = line.trim().split(regex:"\\s+");
    if (p.length == 0) return "ERR vacío";
    String cmd = p[0].toUpperCase(Locale.ROOT);
    try {
        switch (cmd) {
            case "ECHO":
                return joinFrom(p, 1);
            case "TIME":
                return Instant.now().toString();
            case "MAYUS":
                return joinFrom(p, 1).toUpperCase(Locale.ROOT);
            case "SUM":
                if (p.length < 3) return "ERR uso: SUM a b";
                long a = Long.parseLong(p[1]), b = Long.parseLong(p[2]);
                return "SUM=" + (a + b);
            default:
                return "ERR cmd desconocido: " + cmd;
        }
    } catch (Exception e) {
        return "ERR " + e.getMessage();
    }
}

private static String joinFrom(String[] p, int i) {
    StringBuilder sb = new StringBuilder();
    for (int k = i; k < p.length; k++) {

```

```

        if (k > i) sb.append(c: ' ');
        sb.append(p[k]);
    }
    return sb.toString();
}

Run | Debug
public static void main(String[] args) throws Exception {
    int port = (args.length > 0) ? Integer.parseInt(args[0]) : 9001;
    int threads = (args.length > 1) ? Integer.parseInt(args[1]) : 8;
    int M = (args.length > 2) ? Integer.parseInt(args[2]) : 3; // M: capacidad simultánea
    new BackendServer(port, threads, M).start();
}
}

```

3. ClientMulti.java

- Cliente interactivo para enviar solicitudes y mostrar respuestas.
- Permite probar la concurrencia y los distintos servicios.

```

import java.io.*;
import java.net.Socket;
import java.util.Scanner;

public class ClientMulti {
    Run | Debug
    public static void main(String[] args) throws Exception {
        String host = (args.length > 0) ? args[0] : "127.0.0.1";
        int port = (args.length > 1) ? Integer.parseInt(args[1]) : 8080;

        try (Socket s = new Socket(host, port);
            BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));
            PrintWriter out = new PrintWriter(new OutputStreamWriter(s.getOutputStream()), autoFlush:true);
            Scanner sc = new Scanner(System.in)) {

            // Mensaje de bienvenida del backend (a través del LB)
            String welcome = in.readLine();
            if (welcome != null) System.out.println("[SRV] " + welcome);

            System.out.println(x:"Comandos: ECHO, TIME, MAYUS, SUM a b | 'exit' para salir");
            while (true) {
                System.out.print(s:> ");
                String line = sc.nextLine();
                out.println(line);
                if (line.equalsIgnoreCase(anotherString:"exit")) break;
                String resp = in.readLine();
                System.out.println("[SRV] " + resp);
            }
        }
    }
}

```

Este diseño modular facilita escalar el número de backends, servicios y clientes.

5. Resultados

Durante las pruebas se observaron los siguientes resultados:

- El Load Balancer distribuyó correctamente las conexiones entre los backends en forma alternada (round-robin).
- Los backends procesaron correctamente múltiples clientes simultáneos.
- Los servicios (ECHO, TIME, MAYUS, SUM) respondieron de manera confiable y sin errores.
- Al fijar $M=1$ en un backend, se verificó que las peticiones adicionales quedaban en espera, demostrando la efectividad de la sincronización con semáforos.
- Con varios clientes ejecutándose en paralelo, el sistema mantuvo estabilidad y consistencia.

```

Command Prompt - java Bac
C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>javac *.java
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java BackendServer 9001 8 3 & java BackendServer 9002 8 3
[BE 9001] listo. Threads=8 M=3

Scanner sc = new Scanner(System.in);

Command Prompt - java Bac
C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java BackendServer 9002 8 3
[BE 9002] listo. Threads=8 M=3

```

```
Command Prompt - java LoadBalancer x + -
Microsoft Windows [Version 10.0.26100.4349]
(c) Microsoft Corporation. All rights reserved.

C:\Users\miaro>cd documents

C:\Users\miaro\Documents>cd sistemas distribuidos

C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4

C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java LoadBalancer
[LB] Escuchando en 8080 con 2 backends...

Command Prompt - java Client x + -
C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java ClientMulti
[SRV] Backend 9001 OK. Comandos: ECHO, TIME, MAYUS, SUM. 'exit' para salir.
Comandos: ECHO, TIME, MAYUS, SUM a b | 'exit' para salir
> ECHO
[SRV]
>

Command Prompt - java Backend x + -
C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>javac *.java
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java BackendServer 9001 8 3 & java BackendServer 9002 8 3
[BE 9001] listo. Threads=8 M=3
[BE 9001] +/127.0.0.1:55023

Command Prompt - java Client x + -
C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java ClientMulti
[SRV] Backend 9002 OK. Comandos: ECHO, TIME, MAYUS, SUM. 'exit' para salir.
Comandos: ECHO, TIME, MAYUS, SUM a b | 'exit' para salir
> TIME
[SRV] 2025-10-03T21:48:45.003122800Z
>

Command Prompt - java Backend x + -
C:\Users\miaro>cd documents
C:\Users\miaro\Documents>cd sistemas distribuidos
C:\Users\miaro\Documents\Sistemas distribuidos>cd practica 4
C:\Users\miaro\Documents\Sistemas distribuidos\Practica 4>java BackendServer 9002 8 3
[BE 9002] listo. Threads=8 M=3
[BE 9002] +/127.0.0.1:57172
```

6. Conclusión

La práctica permitió reforzar conceptos de multiprocesamiento, multiservicios y balanceo de carga.

Se concluye que:

1. El balanceo de carga es esencial para distribuir equitativamente el trabajo entre servidores y mejorar el rendimiento del sistema.

2. El uso de hilos y pools de ejecución permite que los backends atiendan múltiples clientes sin bloqueos.
3. Los semáforos resultan fundamentales para controlar el acceso a recursos limitados y evitar condiciones de carrera.
4. La arquitectura implementada constituye la base de sistemas reales como clústeres de servidores web, microservicios en la nube y bases de datos distribuidas.

7. Referencias

- Oracle. (n.d.). *Lesson: Concurrency (The Java™ Tutorials > Essential Classes > Concurrency)*. <https://docs.oracle.com/javase/tutorial/essential/concurrency/>
- GeeksforGeeks. (n.d.). *Load Balancer in Distributed Systems*. <https://www.geeksforgeeks.org/load-balancer-in-distributed-systems/>
- Baeldung. (n.d.). *A Guide to Java ExecutorService*. <https://www.baeldung.com/java-executor-service>